



UNIVERSITÀ
DI TORINO

Dipartimento di Informatica

Tecnologie del Linguaggio Naturale

Parte I, Alessandro Mazzei

Federico Chirco

Anno Accademico 2024/2025

Lara is hiring!

Abstract

Il progetto consiste nello sviluppo di un **Dialog System** basato su **Lara Croft**, progettato per interrogare un utente candidato come assistente della celebre archeologa della serie *Tomb Raider*. Il sistema adotta un'architettura modulare composta da un motore di **Natural Language Understanding (NLU)** basato su SpaCy, un **Dialogue Manager (DM)** a stati finiti e un generatore di linguaggio (**NLG**) ibrido. Quest'ultimo combina template deterministici con la realizzazione sintattica dinamica fornita da **SimpleNLG**. I risultati dimostrano come l'integrazione di memoria storica e analisi comportamentale permetta di mantenere la coerenza del personaggio anche in scenari di ambiguità linguistica o risposte parziali, migliorando l'immersività dell'esperienza utente.

1. Introduzione

Questo lavoro presenta la progettazione e l'implementazione di un sistema di dialogo basato su regole che simula un colloquio sotto forma di quiz. Il sistema sottopone l'utente a una serie di domande di diversa tipologia, valuta le risposte fornite, gestisce casi di ambiguità o risposte parziali e fornisce un feedback dinamico durante l'interazione, concludendo infine con una valutazione complessiva delle prestazioni.

In questo contesto, la sfida non è solo comprendere l'intento dell'utente, ma reagire in un modo che sia coerente con il background e il temperamento del personaggio rappresentato.

1.1 Sfide Tecniche

La progettazione di questo assistente ha dovuto affrontare tre problematiche principali:

1. **Robustezza della NLU:** Gestire input utente non strutturati e spesso "sporchi".
2. **Gestione del Dominio:** Implementare una logica di valutazione che non si limiti a risposte "Corretto/Errato", ma che sia in grado di tracciare liste di elementi (domande multiple) e fornire feedback sui progressi parziali.

3. **Varietà del Linguaggio:** Evitare il "fenomeno del disco rotto" tipico dei chatbot tradizionali, utilizzando una memoria storica che impedisca la ripetizione di template identici in successione.
-

2. Requisiti e Installazione

Il sistema è stato sviluppato in **Python**. Il progetto utilizza le seguenti librerie principali, che vanno installate:

- **spaCy** – per l'analisi linguistica (NLU)
 - `pip install spacy`
- **SimpleNLG (Python port)** – per la generazione del linguaggio naturale (NLG)
 - `pip install simplenlg`

Successivamente, è necessario scaricare il modello linguistico inglese di spaCy:

- `python -m spacy download en_core_web_sm`

Una volta completata l'installazione delle dipendenze, il sistema può essere **avviato** con:

- `python main.py`

All'avvio, l'utente viene guidato in una sessione interattiva a riga di comando, durante la quale il sistema gestisce l'intero flusso del colloquio: introduzione, domande del quiz, feedback dinamico e verdetto finale.

3. Architettura del Sistema

L'architettura del sistema di dialogo sviluppato in questo lavoro segue una struttura modulare ispirata al paradigma **NLU-DM-NLG**. Tale suddivisione consente una chiara separazione delle responsabilità, facilitando la manutenibilità del codice e l'estendibilità del sistema.

Il flusso dei dati è ciclico: l'input dell'utente viene prima analizzato dal modulo di comprensione del linguaggio naturale per estrarre il significato, la logica di stato aggiorna il contesto e il generatore traduce le variabili logiche in espressioni naturali.

3.1 Panoramica dei Moduli

Il sistema è composto dai seguenti componenti principali:

- **Natural Language Understanding (NLU):** analizza l'input testuale dell'utente, individuando il tipo di risposta fornita e determinandone la correttezza o l'ambiguità.
- **Dialogue Manager (DM):** gestisce lo stato della conversazione, coordina l'avanzamento del quiz, aggiorna il punteggio e mantiene la memoria della sessione.
- **Natural Language Generation (NLG):** genera le risposte del sistema combinando template predefiniti con frasi realizzate dinamicamente.
- **Main Controller:** coordina l'esecuzione del sistema e gestisce l'interazione con l'utente tramite linea di comando.

3.2 Flusso di Esecuzione

L'interazione tra i moduli avviene secondo il seguente schema:

1. L'utente fornisce un input testuale.
2. Il **Dialogue Manager** inoltra l'input al modulo **NLU** per l'analisi linguistica.
3. Il risultato dell'analisi (corretto, errato, ambiguo o parziale) viene utilizzato dal **Dialogue Manager** per aggiornare lo stato interno del sistema.
4. Il **Dialogue Manager** produce una struttura dati contenente tutte le informazioni necessarie alla generazione della risposta.
5. Il modulo **NLG** utilizza tali informazioni per costruire un output testuale coerente e contestualizzato.
6. La risposta viene restituita all'utente e il ciclo ricomincia fino alla conclusione del quiz.

Questa rappresentazione permette al sistema di trattare in modo uniforme domande di natura diversa, demandando la logica specifica ai moduli di analisi e gestione del dialogo.

4. Rappresentazione dei dati

Il funzionamento del sistema di dialogo si basa su un dataset strutturato di domande, definito all'interno del file **questions.json**. Esso contiene una lista di oggetti JSON, ognuno dei quali fornisce al Dialogue Manager non solo il testo della domanda da visualizzare, ma anche i metadati necessari per validare la risposta e calcolare il punteggio:

- **id**: identificatore univoco della domanda, utilizzato per l'ordinamento e il tracciamento.
- **type**: tipologia della domanda, che determina la strategia di analisi della risposta.
- **topic**: categoria semantica della domanda.
- **question**: testo della domanda presentata all'utente.
- **answer**: risposta corretta, la cui struttura dipende dal tipo di domanda.
- **score**: punteggio associato alla domanda.

4.1 Tipologie di Domande

Il sistema supporta tre tipologie principali di domande, ciascuna delle quali richiede una diversa strategia di interpretazione dell'input utente.

4.1.1 Domande a risposta singola (single)

Le domande a risposta singola richiedono all'utente di fornire un'informazione specifica. La risposta viene considerata corretta se il valore atteso è presente nel testo dell'utente, tenendo conto di eventuali negazioni e utilizzando tecniche di pattern matching per evitare corrispondenze parziali.

4.1.2 Domande booleane (boolean)

Le domande booleane richiedono una risposta di tipo vero/falso. Il sistema analizza l'intento dell'utente rilevando indicatori di affermazione o negazione e segnala eventuali casi di ambiguità, come risposte contraddittorie.

4.1.3 Domande a risposta multipla (multiple)

Le domande a risposta multipla consentono all'utente di fornire più risposte corrette, anche in più turni consecutivi. Il sistema mantiene traccia delle risposte già individuate e assegna il punteggio in modo incrementale.

Questa tipologia introduce una dinamica dialogica più complessa, in cui l'utente può ricevere feedback intermedi sul progresso e completare gradualmente la risposta.

4.2 Ruolo dell'attributo topic

Un elemento centrale della rappresentazione dei dati è il campo **topic**, che associa ogni domanda a una categoria semantica (ad esempio *biography*, *artifacts*, *adventures*).

L'introduzione dei topic risponde a due principali esigenze progettuali:

- **Analisi delle prestazioni:** consente di individuare i punti di forza e di debolezza dell'utente in base agli argomenti trattati.
- **Generazione di feedback avanzato:** il modulo NLG utilizza i topic per produrre commenti finali più mirati e contestualizzati.

In questo modo, il dataset non svolge un ruolo puramente passivo, ma diventa parte integrante del comportamento intelligente del sistema.

5. Natural Language Understanding (NLU)

Il modulo di **Natural Language Understanding (NLU)** ha il compito di interpretare le risposte dell'utente e trasformarle in informazioni strutturate utilizzabili dal Dialogue Manager. In particolare, il sistema deve determinare se una risposta è corretta, errata o ambigua, adattando il comportamento del dialogo in base all'esito.

Nel progetto, l'NLU non mira a una comprensione linguistica "totale", ma implementa un insieme di euristiche robuste e facilmente controllabili, sufficienti per lo scenario del quiz.

5.1 Uso di spaCy

Il sistema utilizza la libreria **spaCy**, utilizzando il modello pre-addestrato `en_core_web_sm`.

In particolare, spaCy viene impiegato per:

- tokenizzazione del testo utente;
- lemmatizzazione (riduzione delle parole alla loro forma base);
- analisi delle dipendenze sintattiche;
- Named Entity Recognition (NER).

L'uso di spaCy consente di lavorare su rappresentazioni linguistiche strutturate anziché su semplici stringhe, permettendo di gestire fenomeni come negazione, ambiguità e variazioni lessicali in modo più affidabile.

5.2 Gestione delle domande a risposta singola

Per le domande che prevedono una risposta univoca, l'NLU opera attraverso un **token matching**. Invece di una ricerca esatta della stringa, il sistema verifica se il token chiave è presente nell'input dell'utente, in modo tale da ignorare parole di contorno.

Inoltre, viene eseguito un **controllo di negazione**: se l'utente inserisce "not" o "no" in una struttura sintattica che nega direttamente la risposta attesa, il sistema considera la risposta errata. Questo riduce casi in cui la risposta corretta è presente nel testo ma usata in forma negata (es. "it's not 1968").

5.3 Gestione delle domande booleane

Le domande booleane (Sì/No) richiedono un'analisi della polarità. Il sistema mappa i token dell'utente su due set di significati:

- **Affermazione:** "yes", "yeah", "correct", "true", ...
- **Negazione:** "no", "nope", "false",

La criticità principale è gestire casi **ambigui**, come frasi che contengono sia elementi affermativi che negativi ("yes... but no...", "true and false"). In quel caso l'NLU restituisce un valore di ambiguità (AMBIGUOUS) e il Dialogue Manager non avanza alla domanda successiva, dando all'utente una possibilità di riformulare.

5.4 Gestione delle domande a risposta multipla

Le domande multiple richiedono che l'utente elenchi più elementi corretti (anche in più turni). Il metodo `_analyze_multiple` si concentra su due aspetti:

1. **estrazione delle risposte presenti** nel testo
 - per ogni elemento atteso nel dataset, verifica se è presente con regex e word boundary
2. **Noise Detection**
 - se l'utente inserisce troppe parole non rilevanti rispetto al numero di risposte trovate, l'input viene considerato ambiguo (es. frasi lunghe con pochissime parole chiave); ciò evita che l'utente possa indovinare inserendo testi lunghissimi carichi di parole a caso.

In caso positivo, l'NLU restituisce la lista di risposte individuate, lasciando al Dialogue Manager il compito di:

- distinguere tra nuove risposte e ripetizioni
- aggiornare lo stato di completamento
- assegnare punti in modo incrementale

6. Dialogue Manager

Il **Dialogue Manager (DM)** rappresenta il cuore logico del sistema: coordina l'interazione tra l'utente e i moduli di Natural Language Understanding (NLU) e Natural Language Generation (NLG), mantenendo lo stato della sessione, gestendo la progressione del quiz e producendo le informazioni necessarie alla generazione delle risposte.

6.1 Session State

Il DM mantiene uno **stato di sessione** (`session_state`) che rappresenta la situazione corrente dell'interazione. Questo stato è condiviso tra i vari turni e aggiornato progressivamente.

Gli attributi principali sono:

- name: nome dell'utente (estratto nella fase di introduzione)
- score: punteggio cumulativo del quiz
- status: stato corrente del sistema

6.1.1 Stati del sistema

Il flusso del dialogo è governato da una **Finite State Machine (FSM)** semplificata, che garantisce che l'utente non possa saltare passaggi logici (es. rispondere al quiz prima di essersi presentato).

Il sistema prevede tre stati principali:

- **INTRO**
Stato iniziale, in cui il sistema accoglie l'utente ed estrae il nome.
- **QUIZ**
Stato centrale dell'interazione, in cui vengono poste le domande e valutate le risposte.
- **END**
Stato finale, attivato quando tutte le domande sono state processate; in questa fase viene generato il verdetto finale.

6.2 Memory Model

Insieme al session state, il Dialogue Manager mantiene una **memoria di interazione** (memory), utilizzata per rendere il dialogo coerente e adattivo.

6.2.1 History

La struttura *history* è una lista che memorizza ogni interazione significativa sotto forma di record contenente:

- domanda posta
- topic associato
- risposta dell'utente
- esito dell'interazione (corretto, errato, ambiguo, ecc.)

Questa cronologia viene utilizzata per:

- analizzare l'andamento recente del quiz
- determinare il "mood" di Lara
- produrre un'analisi finale della performance dell'utente

6.2.2 Used Templates

L'insieme *used_templates* tiene traccia dei template linguistici già utilizzati dal modulo NLG. Questo meccanismo evita di ripetere gli stessi feedback e contribuisce quindi a rendere l'interazione più naturale e meno meccanica, pur restando in un contesto fortemente template-based.

6.3 Estrazione del nome utente

Durante la fase **INTRO**, il Dialogue Manager si occupa dell'estrazione del nome dell'utente.

Il metodo `_handle_intro` utilizza una strategia a più livelli:

1. **pattern espliciti** (“my name is X”, “I’m X”, “call me X”), tramite regex
2. **Named Entity Recognition (NER)**, cercando entità PERSON nel testo
3. **filtri di sicurezza**, ad esempio ignorare nomi legati a “Lara” o “Croft” quando l’utente li cita nella frase

Se il nome viene riconosciuto con successo, viene salvato nello stato di sessione e utilizzato nelle risposte successive. In caso contrario, viene assegnato un valore di fallback (“Candidate”), permettendo comunque la prosecuzione dell’interazione.

6.4 Interazione logica del quiz

Il metodo `_handle_quiz` rappresenta la parte più articolata del Dialogue Manager. Il suo compito è trasformare l’output dell’NLU in decisioni dialogiche concrete.

6.4.1 Gestione delle risposte parziali

Per le domande a risposta multipla, il DM mantiene nello stato di sessione:

- `found_answers`: risposte corrette già individuate
- `expected_answers`: numero totale di risposte attese

Questo consente al sistema di:

- riconoscere nuove risposte valide
- ignorare ripetizioni
- distinguere tra completamento riuscito e risposta ancora incompleta

6.4.2 Gestione dell’ambiguità

Il Dialogue Manager tiene traccia di un contatore di ambiguità (`ambiguity_count`).

Se l’utente fornisce più volte risposte ambigue consecutive (es. troppo rumore nelle domande a risposta multipla o risposte contraddittorie nelle domande booleane), il sistema:

- inizialmente chiede una riformulazione
- successivamente forza l’avanzamento alla domanda successiva, marcando la risposta come errata

Questa strategia impedisce che l’interazione rimanga bloccata su una singola domanda.

6.4.3 Avanzamento delle domande

L’avanzamento alla domanda successiva avviene solo quando:

- la risposta è corretta o errata
- la risposta multipla è completata
- il numero massimo di ambiguità è stato superato

6.4.4 Calcolo del punteggio

Il punteggio viene aggiornato in modo diverso a seconda del tipo di domanda:

- **domande singole o booleane**: il punteggio viene assegnato interamente in caso di risposta corretta

- **domande multiple** : il punteggio totale viene suddiviso tra le singole risposte corrette, consentendo all'utente di accumulare punti progressivamente man mano che individua gli elementi richiesti

6.5 Analisi del mood

Il DM analizza la cronologia recente delle risposte per determinare il **mood** di Lara.

In particolare, vengono osservati gli ultimi due esiti:

- due risposte corrette di fila → mood **POSITIVE**
- due risposte errate di fila → mood **NEGATIVE**
- altri casi → mood **NEUTRAL**

Il mood non influisce sulla logica del quiz, ma viene passato al modulo NLG per **adattare il tono delle risposte**, rendendo Lara Croft più incoraggiante o più severa a seconda del comportamento dell'utente.

6.6 Analisi della performance e calcolo del verdetto

Al termine del quiz, il Dialogue Manager analizza la cronologia delle risposte per individuare:

- il **topic con più errori** (worst topic)
- il **topic con migliori risultati** (best topic)

Queste informazioni vengono utilizzate per generare un feedback finale personalizzato.

Infine, il verdetto conclusivo (**PERFECT, PASSED, FAILED**) viene calcolato confrontando il punteggio ottenuto con il punteggio massimo teorico, utilizzando delle soglie prefissate. Il risultato viene passato al modulo NLG, che produce la risposta finale dell'intervista.

7. Natural Language Generation (NLG)

Il modulo di **Natural Language Generation (NLG)** rappresenta la fase finale della pipeline, dove le informazioni prodotte dal Dialogue Manager (decisioni logiche, punteggi numerici) vengono tradotti in un linguaggio naturale coerente.

La sfida principale in questa sezione è stata bilanciare la precisione grammaticale con il mantenimento della personalità di Lara Croft.

7.1 Approccio ibrido: template + SimpleNLG

La generazione linguistica è affidata alla classe NLGEngine, che adottando una strategia ibrida combina due approcci complementari:

- i **template**, progettati per riflettere il carattere di Lara Croft e il suo tono (diretto, sarcastico, severo);
- **SimpleNLG**, tramite **generazione dinamica**, produce frasi sensibili ai dati (ad esempio progressi, punteggi o report finali).

7.1.1 Gestione dei template unici

Per evitare che Lara ripeta la stessa frase più volte, il sistema mantiene un insieme di **template già utilizzati** (used_templates).

Il metodo `pick_unique_template` seleziona preferibilmente un template non ancora usato; solo nel caso in cui tutte le opzioni siano già state utilizzate, la memoria viene parzialmente resettata.

7.1.2 Generazione dinamica con SimpleNLG

SimpleNLG viene utilizzato per generare frasi che dipendono da valori dinamici, in particolare:

- **progresso nelle domande a risposta multipla** (es: *"You have identified 2 out of 3 items."*)
- **report di performance finale**, che combina successi ed errori per topic
- **report del punteggio**, con gestione automatica di tempo verbale, plurale e modificatori

L'uso di SimpleNLG consente di delegare al motore linguistico aspetti come:

- accordo singolare/plurale
- gestione dei tempi verbali
- costruzione di frasi subordinate e coordinate

riducendo il rischio di errori tipici della generazione manuale tramite stringhe.

7.2 Coordinamento della generazione (`generate_response`)

Il metodo `generate_response` rappresenta il punto di integrazione tra DM e NLG.

Riceve in ingresso una struttura dati (`dm_data`) contenente:

- tipo di messaggio (introduzione, feedback, fine quiz)
- esito dell'interazione
- mood corrente
- progresso e punteggio
- informazioni di memoria

In base a questi dati, il metodo:

1. seleziona il set corretto di template;
2. ne sceglie uno coerente e non ripetuto;
3. integra eventuali frasi dinamiche generate con SimpleNLG;
4. restituisce:
 - la risposta completa da mostrare all'utente;
 - il template atomico utilizzato, per l'aggiornamento della memoria.

Questa separazione permette di mantenere la memoria linguistica indipendente dalla forma finale del messaggio.

7.3 Generazione del verdetto finale

Al termine del quiz, il modulo NLG genera una risposta articolata composta da:

- un commento sulla performance complessiva (best/worst topic);
- una frase che riassume il punteggio ottenuto;

- un verdetto finale coerente con l'esito dell'intervista.

8. Valutazione

8.1 Analisi comparativa dei dialoghi

Per valutare il comportamento del sistema dialogico in condizioni realistiche, sono stati analizzati tre dialoghi rappresentativi di profili di candidato differenti: un candidato ideale, uno esitante e uno incompetente. I tre dialoghi coprono **tutti i casi gestiti dal progetto**, inclusi: risposte corrette, ambigue ed errate, completamento progressivo delle domande multiple, gestione dell'ambiguità, variazione del mood e generazione di report finali personalizzati.

8.1.1 L'archeologo ideale

Il primo dialogo rappresenta il **caso ideale**, in cui il candidato risponde correttamente a tutte le domande.

```
LARA: Welcome. I'm Lara Croft. Many aspire to be my assistants,
      but few possess the necessary preparation. Let's start with your name.
YOU: my name is federico

LARA: An interesting name, Federico. I hope your skills live up to it.

LARA: First question. In what year was I born?
YOU: it was 1968, i'm sure

LARA: Exactly. Just as I remember it.

LARA: Listen closely: My father was not also an archaeologist.
YOU: false, he was a famous archaeologist

LARA: Not bad at all! Keep this up and I might actually trust you.

LARA: Next question: Which artifacts did I recover in Tomb Raider II?
YOU: the dagger of xian and the mask of tornarsuk

LARA: Excellent! You surprise me; I wouldn't have bet on you. You have identified 2 out of 2 items.

LARA: Moving on: Which city did I explore in search of the Philosopher's Stone?
YOU: i believe it was rome

LARA: Listen closely: The Dagger of Xian grants immortality to whoever uses it correctly.
YOU: yes, it grants immortality

LARA: Impressive, you're a true expert on my history!

LARA: Listen closely: Which locations did I visit during my early expeditions?
YOU: peru, greece and egypt

LARA: Correct. Perhaps there's hope for you after all. You have identified 3 out of 3 items.
LARA: You grasped the nuances of my past expeditions.
LARA: You achieved 38 point that which is a perfect result.

LARA: Exceptional. It's rare to find someone on my level. Welcome to the team.
```

Fin dalla fase introduttiva, il sistema riesce a **estrarre correttamente il nome dell'utente** grazie al pattern esplicito *"my name is"*, permettendo a Lara di personalizzare il dialogo.

Nella fase di quiz, il candidato:

- risponde correttamente alle **domande a risposta singola**, anche usando forme discorsive ("it was 1968, I'm sure");

- gestisce correttamente le **domande booleane**, esplicitando chiaramente l'intenzione ("false");
- completa correttamente le **domande multiple**, fornendo tutte le risposte attese in un'unica interazione.

Il sistema riconosce le risposte senza ambiguità, assegna il punteggio massimo e attiva uno **streak positivo**, che influenza positivamente il *mood* di Lara. Questo si riflette direttamente nel feedback, che diventa progressivamente più incoraggiante.

Nel verdetto finale, il NLG genera:

- un **commento positivo sulle competenze tematiche** ("You grasped the nuances of my past expeditions");
- un **resoconto del punteggio** che evidenzia il risultato perfetto;
- una frase di assunzione coerente con il verdetto PERFECT.

8.1.2 Il candidato titubante

Il secondo dialogo rappresenta un candidato con **conoscenze parziali e comportamento incerto**, utile per mostrare la gestione dell'ambiguità e delle risposte incomplete.

```
LARA: Welcome. I'm Lara Croft. Many aspire to be my assistants,
      but few possess the necessary preparation. Let's start with your name.
YOU: hi lara

LARA: You didn't introduce yourself. I'll settle for this: Candidate.

LARA: First question. In what year was I born?
YOU: 1968

LARA: Exactly. Just as I remember it.

LARA: Listen closely: My father was not also an archaeologist.
YOU: no that's not true

LARA: First you affirm, then you deny... Be consistent, Candidate.
YOU: false

LARA: Good. You didn't let yourself be fooled.

LARA: Next question: Which artifacts did I recover in Tomb Raider II?
YOU: mmmh the dagger of xian

LARA: You know there's more to it. Say it. You have identified 1 out of 2 items.
YOU: and the mask of tornarsuk too

LARA: You've added the missing pieces. Well done. You have identified 2 out of 2 items.

LARA: Next question: Which city did I explore in search of the Philosopher's Stone?
YOU: maybe london?

LARA: Incorrect. You should study my journals more closely.

LARA: Listen closely: The Dagger of Xian grants immortality to whoever uses it correctly.
YOU: yes or no ... i don't remember

LARA: It's a simple question: true or false? Stop contradicting yourself.
YOU: nope

LARA: Listen closely: Which locations did I visit during my early expeditions?
YOU: peru, bolivia, ecuador, cyprus, madrid, greece, london, paris, venice, venezuela and maybe egypt
```

```

LARA: I don't have time for this chatter. List the items clearly.
YOU: peru, greece

LARA: Interesting... but incomplete. You have identified 2 out of 3 items.
YOU: venezuela

LARA: Another mistake? I'm starting to lose my patience. You have identified 2 out of 3 items.
LARA: You grasped the nuances of ancient and dangerous relics but you committed 3 errors regarding my past expeditions.
LARA: You scored 26 point that which is a solid start.

LARA: Sufficient. I'll take you with me, but try not to slow me down.

```

Durante l'introduzione, il candidato non fornisce un nome esplicito. Il sistema, non riuscendo a estrarre un'entità valida, assegna correttamente il placeholder "*Candidate*".

Nel corso del quiz emergono diversi fenomeni rilevanti:

- una **domanda booleana ambigua**, in cui l'utente esprime prima un'affermazione e poi una negazione ("yes or no ..."), attivando il meccanismo di *AMBIGUOUS_BOOL*;
- un **completamento progressivo** delle domande multiple, con feedback intermedi ("You have identified 1 out of 2 items");
- una **domanda multipla con rumore eccessivo**, che porta a un caso di *AMBIGUOUS_MULTI* e richiede una risposta più concisa.

Nel verdetto finale, il sistema genera un report bilanciato:

- riconosce un punto di forza ("ancient and dangerous relics"),
- evidenzia il tema problematico ("past expeditions"),
- assegna un verdetto *PASSED* con una formulazione coerente.

8.1.3 L'incompetente

Il terzo dialogo rappresenta il **caso limite negativo**, progettato esplicitamente per mostrare il comportamento del sistema quando l'utente fornisce **risposte ripetutamente errate o ambigue**, superando la soglia di tolleranza prevista.

```

LARA: Welcome. I'm Lara Croft. Many aspire to be my assistants,
      but few possess the necessary preparation. Let's start with your name.
YOU: i don't want to tell you

LARA: A name would have been useful. For now, you are 'Candidate'.

LARA: First question. In what year was I born?
YOU: 1990

LARA: Absolutely not. Focus, Candidate.

LARA: Tell me: My father was not also an archaeologist.
YOU: yeah

LARA: Another mistake? I'm starting to lose my patience.

LARA: Moving on: Which artifacts did I recover in Tomb Raider II?
YOU: i don't remember

LARA: Another failure. If we were in the field, you'd be in serious trouble by now. You have identified 0 out of 2 items.

LARA: Tell me: Which city did I explore in search of the Philosopher's Stone?
YOU: london

LARA: This is embarrassing... do you know anything about me at all?

```

LARA: Next question: The Dagger of Xian grants immortality to whoever uses it correctly.
 YOU: yes or no, i'm not sure

LARA: Make up your mind. Is it a yes or a no? I can't stand indecision.
 YOU: maybe yes, maybe not

LARA: First you affirm, then you deny... Be consistent, Candidate.
 YOU: yes no bla bla

LARA: No. Disappointing.

LARA: Tell me: Which locations did I visit during my early expeditions?
 YOU: peru, then ...

LARA: That's not all. Keep going. You have identified 1 out of 3 items.
 YOU: isn't peru enough?

LARA: Something is still missing. Think harder. You have identified 1 out of 3 items.
 YOU: peru

LARA: Another failure. If we were in the field, you'd be in serious trouble by now. You have identified 1 out of 3 items.
 LARA: You committed 6 errors regarding my personal life and family.
 LARA: You scored 4 only point that which is insufficient.

LARA: Disastrous. Come back when you've actually read a history book. This interview is over.

Un punto centrale di questo dialogo è la gestione dell'**ambiguità reiterata**:

- in una *domanda booleana*, l'utente fornisce delle risposte successive ambigue o contraddittorie (*yes no*)
- in una *domanda a risposta multipla*, l'utente fornisce una sola risposta corretta (*peru*) e poi ripete informazioni già note o tenta di chiudere prematuramente la risposta.

In entrambi i casi, il Dialogue Manager incrementa progressivamente il contatore di ambiguità. Superata la soglia massima prevista (tre tentativi), il sistema:

- forza la valutazione come risposta errata;
- interrompe la possibilità di ulteriori chiarimenti;
- attiva un feedback negativo più severo.

Nel verdetto conclusivo, il NLG genera un report fortemente negativo:

- l'analisi della performance individua esclusivamente errori, senza alcun punto di forza tematico;
- il punteggio estremamente basso viene enfatizzato linguisticamente tramite l'avverbio *only*;
- la frase di chiusura è coerente con il verdetto *FAILED*, sancendo l'esclusione del candidato.

8.2 Trindi Ticklist

La Trindi Tricklist fornisce un insieme di criteri utili a valutare la bontà di un sistema di dialogo.

1. L'interpretazione è relativa al contesto?

Sì, parzialmente.

L'interpretazione delle risposte dell'utente è fortemente dipendente dal **contesto della domanda corrente**, gestito dal Dialogue Manager tramite lo stato della sessione e l'indice della domanda attiva. Il modulo NLU non interpreta l'input in modo isolato, ma utilizza il tipo della domanda (single, boolean, multiple) e la risposta attesa.

2. Il sistema può gestire risposte che forniscono più informazioni di quelle richieste?

Sì.

Questo caso è esplicitamente gestito:

- nelle domande singole, attraverso token matching;
- nelle domande booleane, risposte verbose vengono correttamente ridotte all'intenzione principale (affermazione o negazione);
- nelle domande multiple, il sistema estrae solo le risposte rilevanti, ignorando il resto del contenuto.

3. Il sistema può gestire risposte che forniscono informazioni diverse da quelle attese?

Parzialmente.

Se l'utente fornisce informazioni semanticamente scorrette o non pertinenti:

- nelle domande singole, l'assenza della risposta attesa porta a un esito errato;
- nelle domande multiple, risposte non appartenenti all'insieme target vengono ignorate;
- nelle booleane, risposte non riconducibili a una polarità valida vengono trattate come non valide o ambigue.

Il sistema non tenta di reinterpretare informazioni alternative, considerando errate risposte di questo tipo.

4. Il sistema può gestire risposte che forniscono meno informazioni di quelle richieste?

Sì.

Questo è uno dei casi gestiti nelle domande a risposta multipla:

- il sistema supporta **risposte parziali**, aggiornando lo stato con le risposte già individuate;
- viene fornito un feedback esplicito di incompletezza;
- l'utente può completare la risposta in più turni, fino al completamento o al fallimento.

5. Il sistema può gestire risposte ambigue?

Sì.

L'ambiguità è gestita distinguendo:

- rilevazione di contraddizioni (es. "yes and no") nelle risposte alle domande booleane;
- eccesso di rumore linguistico nelle risposte multiple.

In caso di ambiguità reiterata, il Dialogue Manager applica una **soglia di tolleranza**, oltre la quale la risposta viene considerata errata.

6. Il sistema può gestire negazioni?

Parzialmente.

Le negazioni sono gestite in modo esplicito nelle domande singole con l'utilizzo del dependency parsing di spaCy per identificare se la negazione è direttamente legata alla risposta attesa.

7. Il sistema può gestire l'assenza di risposta?

Sì.

Input vuoti vengono intercettati nel main:

- il sistema non avanza nel dialogo;
 - viene richiesto esplicitamente di rispondere.
-

8. Il sistema può gestire input contenenti “rumore”?

Sì, nel caso delle domande a risposta multipla come espresso nelle precedenti domande.

9. Il sistema può gestire sotto-dialoghi iniziati dall'utente?

No.

Il sistema non supporta sotto-dialoghi o iniziative autonome dell'utente.
Ogni input è interpretato esclusivamente come risposta alla domanda corrente.

10. Il sistema chiede solo domande adeguate?

Sì.

Le domande sono predefinite e presentate in sequenza.

11. Il sistema può gestire informazioni inconsistenti?

Sì.

Informazioni inconsistenti (es. “yes... no... maybe”) vengono rilevate come ambigue.
Se l'inconsistenza persiste oltre la soglia, il sistema forza una valutazione negativa e prosegue.

12. Il sistema può gestire una revisione delle credenze?

No.

Il sistema non mantiene un modello di credenze aggiornabile nel tempo.
Ogni risposta è valutata localmente rispetto alla domanda corrente, senza possibilità di revisione esplicita.

13. Il sistema controlla la sua comprensione delle intenzioni dell'utente?

Parzialmente.

Il sistema:

- segnala esplicitamente l'ambiguità;
- chiede di riformulare o chiarire.

Tuttavia, non implementa meccanismi di conferma esplicita ("*Did you mean...?*"), limitandosi a feedback indiretti.

9. Conclusioni

Lo sviluppo di questo progetto ha permesso di comprendere meglio quanto sia complesso interpretare correttamente il linguaggio naturale. Anche frasi apparentemente semplici possono assumere significati diversi a seconda della loro formulazione, del contesto e delle intenzioni dell'utente. Durante l'implementazione del sistema è emerso come gestire tutte queste varianti sia tutt'altro che banale.

Molte delle soluzioni adottate funzionano bene in situazioni specifiche, ma mostrano dei limiti quando l'input dell'utente si discosta anche leggermente dai casi previsti. Per coprire un numero sempre maggiore di situazioni è spesso necessario introdurre nuove regole, rendendo il sistema più complesso e difficile da mantenere. Questo evidenzia quanto sia complicato affrontare la varietà del linguaggio umano utilizzando esclusivamente approcci basati su regole fisse.

Il progetto mette quindi in luce sia le potenzialità sia i limiti di un sistema deterministico di comprensione del linguaggio, mostrando come la flessibilità del linguaggio naturale rappresenti una sfida significativa per questo tipo di soluzioni.